

BERT

Bidirectional Encoder Representations from Transformers

Devlin, Chang, Lee, Toutanova

Google AI Language · October 2018 · NAACL 2019

WHAT YOU'LL LEARN

- Why one-directional language models weren't enough for understanding
- Masked Language Modeling and the bidirectional context insight
- BERT's encoder-only architecture and the role of [CLS], [SEP]
- Token, segment, and position embeddings — the three-way sum
- The fine-tuning paradigm BERT defined: one model, many task heads
- The BERT family: RoBERTa, DistilBERT, ELECTRA, DeBERTa, ModernBERT
- Why encoder models still power production search and retrieval

1. Overview

What BERT is, in one paragraph

BERT — Bidirectional Encoder Representations from Transformers — is a language model published by Google AI Language in October 2018. It used only the *encoder* half of the Transformer architecture (from the 2017 "Attention Is All You Need" paper) and was pretrained on a huge corpus of unlabeled text using two clever objectives: predict randomly masked words from their context, and predict whether two sentences appeared next to each other in the original text. After this pretraining, BERT could be fine-tuned on small labeled datasets to achieve state-of-the-art results on a wide range of NLP tasks — classification, named entity recognition, question answering, sentence similarity. BERT defined the "pretrain then fine-tune" paradigm that dominated NLP from 2018 to roughly 2022.

Why it matters

BERT was the model that proved transfer learning *works* for NLP at scale. Before BERT, every NLP task required its own labeled dataset and its own custom model architecture. After BERT, the standard recipe became: download the pretrained model, attach a small task-specific head, fine-tune on whatever labeled data you have. Quality jumped dramatically across the board.

On the GLUE benchmark — the standard suite of language understanding tasks at the time — BERT immediately beat the previous best by about 10 points on release. Researchers and practitioners spent the next several years building on the BERT recipe.

Even today, with decoder-only LLMs (GPT, Claude, Llama) dominating consumer-facing applications, BERT-style encoder models quietly power a large fraction of production NLP — search ranking at Google and Bing, embedding models for RAG systems, named entity recognition in enterprise tools, content moderation. When you see "sentence-transformers" or "all-MiniLM-L6-v2" or "BGE", you're looking at a BERT descendant.

Where BERT fits in the bigger story

From the LLM Evolution companion guide, you may recall the "BERT and GPT-1" fork in 2018. Both papers used the new Transformer architecture, but they made opposite choices about which half to keep:

GPT kept the decoder — left-to-right, autoregressive, generation-focused. This lineage led to GPT-2, GPT-3, ChatGPT, and modern LLMs.

BERT kept the encoder — bidirectional, understanding-focused. This lineage led to RoBERTa, DeBERTa, sentence embeddings, and the entire "reading comprehension" branch of NLP.

Both lineages exist today. The decoder lineage is what most people mean by "LLM" in 2026. The encoder lineage is what most production systems silently rely on for the dozens of small NLP decisions that happen between the user and the LLM.

How to read this guide

Section 2 covers why pretraining-then-fine-tuning was necessary in 2018. Section 3 covers BERT's central insight: bidirectional context via masked language modeling. Section 4 walks through the architecture. Section 5 covers the pretraining setup. Section 6 covers the fine-tuning paradigm. Section 7 has code. Section 8 covers limitations and the family of follow-up models. Section 9 covers real-world adoption. Section 10 has worked examples. Section 11 has interview Q&A.;

Prerequisites

You should be comfortable with the Transformer architecture — specifically self-attention, the Q/K/V mechanism, and what "encoder" means in that context. If those terms feel fuzzy, read the [Attention Is All You Need](#) companion guide first. Some familiarity with cross-entropy loss and the general shape of supervised learning is useful but not required.

2. The Problem: Language Understanding in 2018

Going into 2018, NLP was in a frustrating place. The Transformer had shown that attention worked for machine translation. Word embeddings (Word2Vec, GloVe) gave useful word-level representations. But applying these advances to *specific downstream tasks* — classifying sentences, extracting entities, answering questions — still required substantial engineering and lots of labeled data per task.

2.1 The dominant pattern: one model per task

For each NLP task, you'd build a separate model. A sentiment classifier was its own LSTM with its own training run on a labeled sentiment dataset. A named entity recognizer was a different LSTM with a different head, trained on annotated NER data. A question-answering system was yet another architecture.

Each of these required:

1. **A labeled dataset** for the specific task. Often the bottleneck — labeling sentiment is doable; labeling thousands of QA pairs or NER spans is expensive.
2. **A task-specific architecture**, often heavily engineered with features, rule-based components, and task-specific tricks.
3. **Training from scratch** (or from word embeddings) for each task. No shared knowledge transferred across tasks.

2.2 ELMo and ULMFiT: early hints

By early 2018, two papers had started chipping at this. ELMo (Peters et al., Allen AI, early 2018) trained a deep bidirectional LSTM on a language modeling objective, then used its hidden states as **contextual word embeddings** for downstream tasks. ULMFiT (Howard & Ruder, May 2018) showed that you could pretrain a language model, then fine-tune the whole thing on a target task, with big gains.

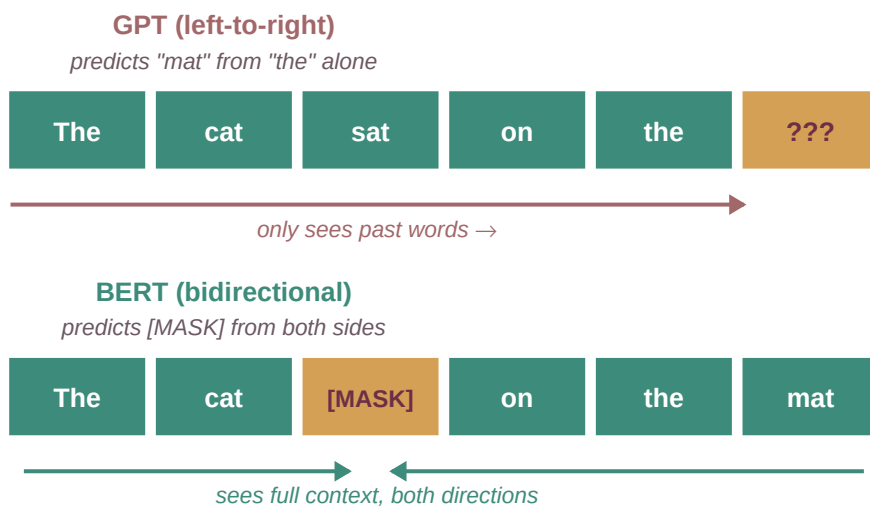
Both pointed at the same idea: *language understanding* could be transferred from a generic pretraining objective to a specific task. But neither used the Transformer, and both had limitations — ELMo's bidirectionality was a concatenation of two separate one-way LSTMs, not truly bidirectional; ULMFiT's language modeling was strictly left-to-right.

2.3 GPT-1 and the left-to-right limitation

In June 2018, OpenAI published GPT-1 (Radford et al.). It used the Transformer decoder, pretrained on unlabeled text with a standard left-to-right language modeling objective (predict the next word). It could be fine-tuned for downstream tasks with reasonable success.

But GPT-1 had a fundamental architectural constraint: **each word could only see the words before it**. The causal mask in the decoder forced left-to-right attention. This is fine — even necessary — for generation. But for many understanding tasks, it's a handicap.

Left-to-right vs bidirectional: what each model sees



BERT can use words AFTER the target to predict it.

This is the central insight: understanding benefits from bidirectional context.

Figure 2.1 — Left-to-right models can only use words that appeared earlier in the sentence. BERT removes this restriction by using bidirectional context.

Consider the sentence "The bank on the river was muddy." To understand what "bank" means — the financial institution or the river bank — you need to see the words that come *after* it. A left-to-right model, when processing "bank", sees only "The". It has to guess. A bidirectional model sees "on the river was muddy" and can resolve the ambiguity.

THE QUESTION DEVLIN ET AL. ASKED

If understanding tasks benefit from seeing both left and right context, can we pretrain a Transformer encoder that uses bidirectional attention — but how do we define a pretraining objective that won't trivially leak the answer? In standard language modeling, if you let the model see both directions, then when predicting word t it would just look at word t in the input. Pointless.

3. The Key Ideas: MLM and NSP

BERT's central contributions are two pretraining objectives that let a bidirectional model learn meaningful representations from unlabeled text. They're elegant once you see them.

3.1 Masked Language Modeling (MLM)

The breakthrough trick: instead of asking the model to predict the next word, randomly **mask** some of the words in the input and ask the model to predict what they should be. Because the to-be-predicted word is replaced with a special [MASK] token, the model can't trivially copy it — it has to use the surrounding context (from both sides) to figure out what the masked word should be.

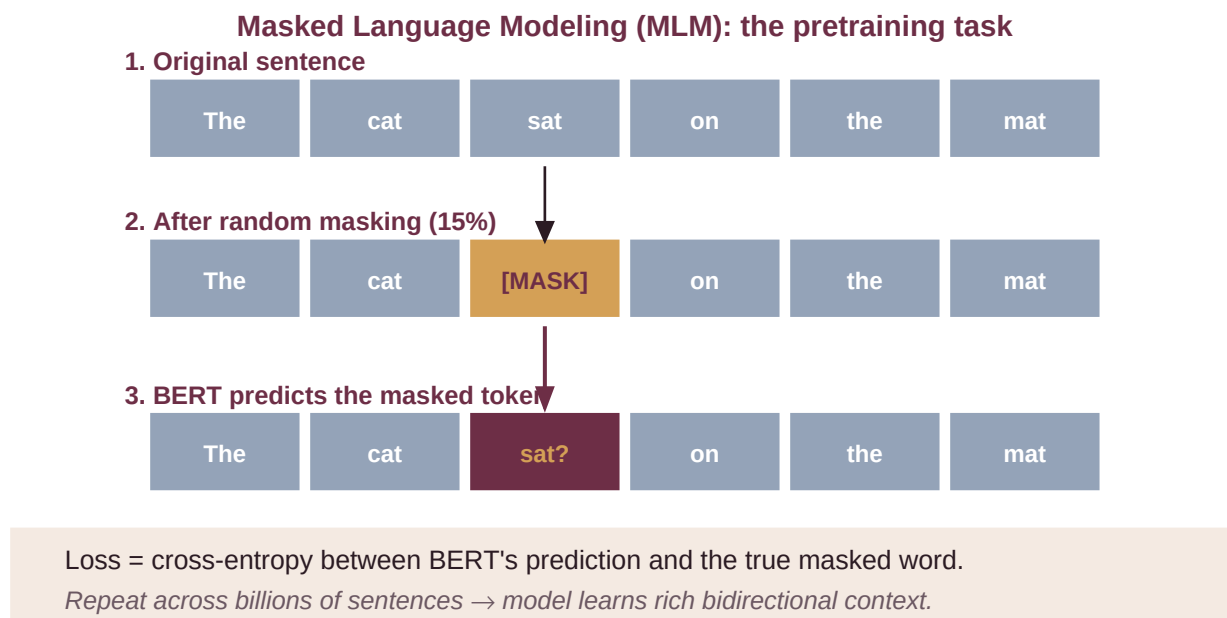


Figure 3.1 — Masked Language Modeling. A random 15% of input tokens are replaced with [MASK]. The model must predict the original token using the surrounding bidirectional context.

The actual masking strategy

BERT doesn't simply replace all selected tokens with [MASK]. There's a subtlety: if the model only ever saw [MASK] tokens at training time, it would never learn good representations for normal tokens (since at fine-tuning time, there are no [MASK] tokens). The paper's solution:

Pick 15% of input tokens. Of those:

- 80% are replaced with [MASK].
- 10% are replaced with a random other token.
- 10% are left unchanged.

In all three cases, the model is asked to predict the *original* token. The random-replacement and unchanged cases force the model to maintain good representations for every token, not just the [MASK] one. This is a small detail with significant practical impact.

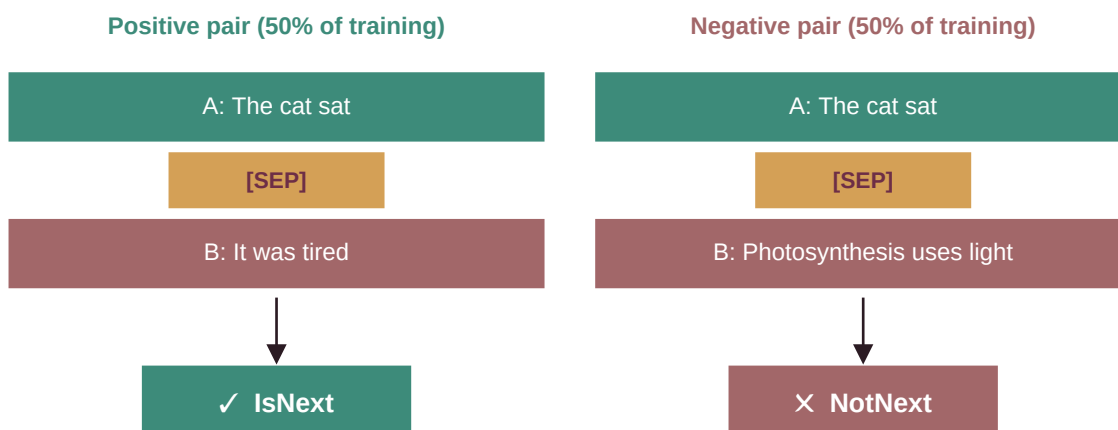
WHY 15%?

Empirically, 15% strikes a balance. Lower masking rates give less signal per training example (the model only learns from a few tokens out of many). Higher masking rates make it too hard — if you mask 50% of words, there's not enough context to predict any of them reliably. Recent research (Wettig et al. 2023) has questioned whether 15% is optimal at all scales; modern models often experiment with higher rates.

3.2 Next Sentence Prediction (NSP)

BERT's second pretraining objective was Next Sentence Prediction. Take two sentences A and B. With 50% probability, B is the actual sentence that followed A in the source document. With 50% probability, B is a random sentence from somewhere else in the corpus. The model is asked to predict which case it's in.

Next Sentence Prediction (NSP): does B follow A?



BERT learns sentence-level relationships, useful for QA and inference tasks.

Figure 3.2 — Next Sentence Prediction. BERT learns to distinguish actual next sentences from random ones, picking up sentence-level relationships in the process.

NSP was meant to teach BERT about sentence-level relationships — useful for downstream tasks like question answering and natural language inference, which involve reasoning across sentence boundaries.

WAIT — DOES NSP ACTUALLY WORK?

Later research showed NSP isn't actually very useful. RoBERTa (Meta, 2019) removed NSP and found that BERT-style models trained without it performed *better* on downstream tasks — the NSP signal was too easy to learn and didn't really teach meaningful sentence-level reasoning. Most modern BERT-family models skip NSP entirely. But the original BERT paper used it, so it's part of the standard description of the architecture.

3.3 The combined objective

BERT was trained on both objectives simultaneously: a batch of input passes through the model, the MLM loss is computed for the masked positions, the NSP loss is computed for the [CLS] token, and they're added

together. Both signals flow back through the same network, shaping a single set of weights.

The combined objective is what BERT actually optimized:

$$L_{\text{total}} = L_{\text{MLM}} + L_{\text{NSP}}$$

4. Architecture

BERT is structurally simpler than the original Transformer because it uses only the encoder. There's no decoder, no cross-attention, no causal mask. Just a stack of transformer encoder layers operating on bidirectional self-attention.

4.1 Two model sizes

The paper released two configurations:

	BERT-Base	BERT-Large
Layers (L)	12	24
Hidden size (H)	768	1024
Attention heads	12	16
Parameters	110M	340M

BERT-Base was the standard for most research and production use. BERT-Large was used to claim state-of-the-art results on benchmarks. Both follow the same design principles — just different depth and width.

4.2 The full stack

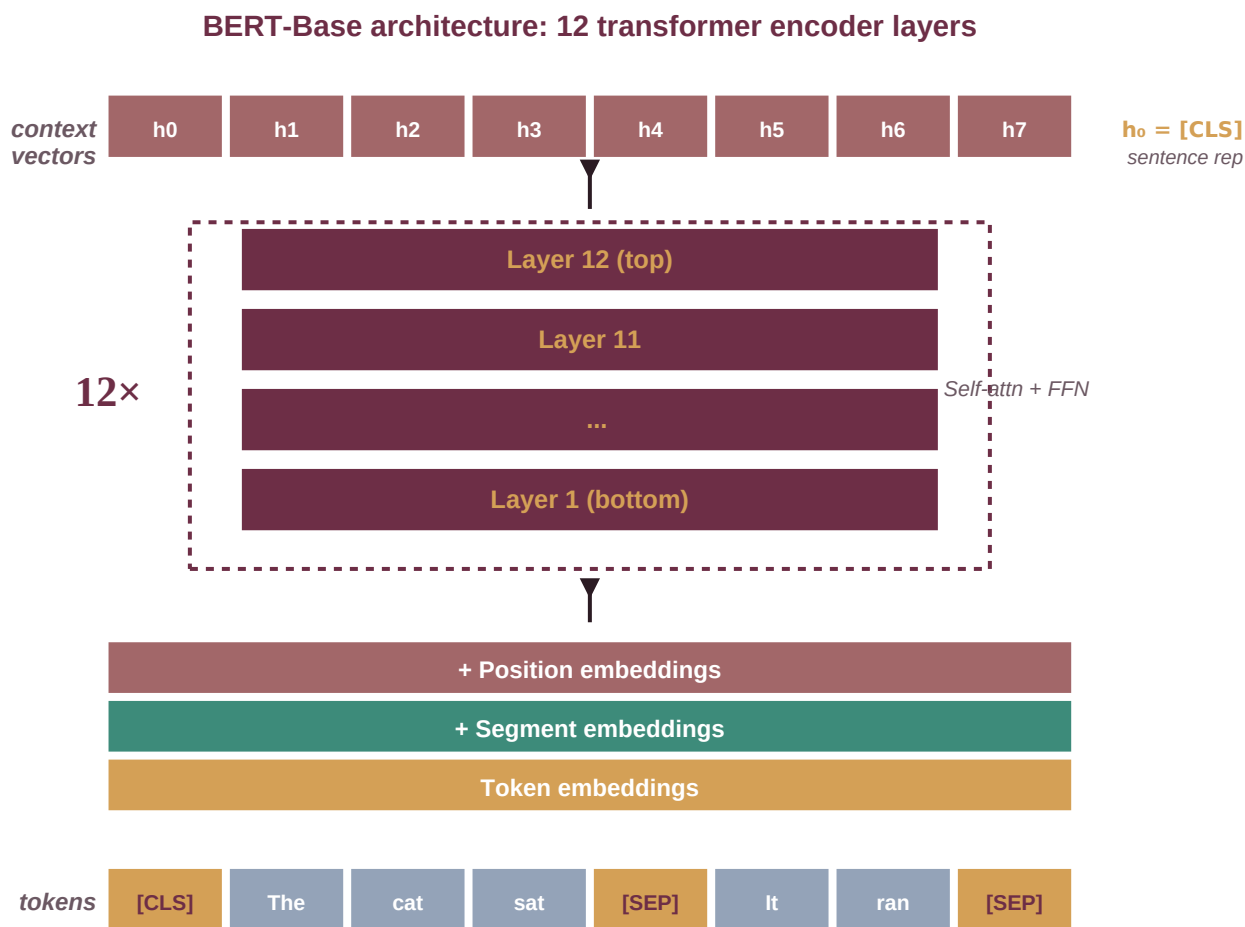


Figure 4.1 — BERT-Base architecture. Tokens enter at the bottom, get embedded (with three additive components), pass through 12 transformer encoder layers, and produce one contextual vector per input position at the top.

Reading the figure bottom-up:

- 1. Tokenized input.** The text is split into subword tokens using WordPiece. Special tokens are added: [CLS] at the start, [SEP] separating sentence pairs.
- 2. Three embeddings, summed.** Each position gets a token embedding (what the word is), a segment embedding (which sentence it belongs to), and a position embedding (where it is in the sequence). These three vectors are summed element-wise.
- 3. Twelve transformer encoder layers.** Each layer is a standard transformer block: multi-head self-attention, then a feed-forward network, each wrapped in residual connections and layer normalization. The self-attention is fully bidirectional — every position attends to every other position, no masking.
- 4. Contextual output vectors.** One vector per input position, each containing information from across the entire sequence. The first position (corresponding to [CLS]) is special — it's typically used as a sentence-level representation.

4.3 Special tokens

[CLS] — the classification token

Every BERT input starts with the special [CLS] token. Its corresponding output vector — the first one in the output sequence — is the standard place to pull a single "summary of the input" representation. For classification tasks, you put a small head on top of this vector. The model learns during pretraining (via NSP) and fine-tuning to make this vector useful as a global summary.

[SEP] — the separator token

Used to separate two sentences in inputs that have two parts (e.g., question and passage, or premise and hypothesis). Also placed at the very end of the input. Combined with segment embeddings, this tells the model where one sentence ends and the next begins.

[MASK] — the mask token (pretraining only)

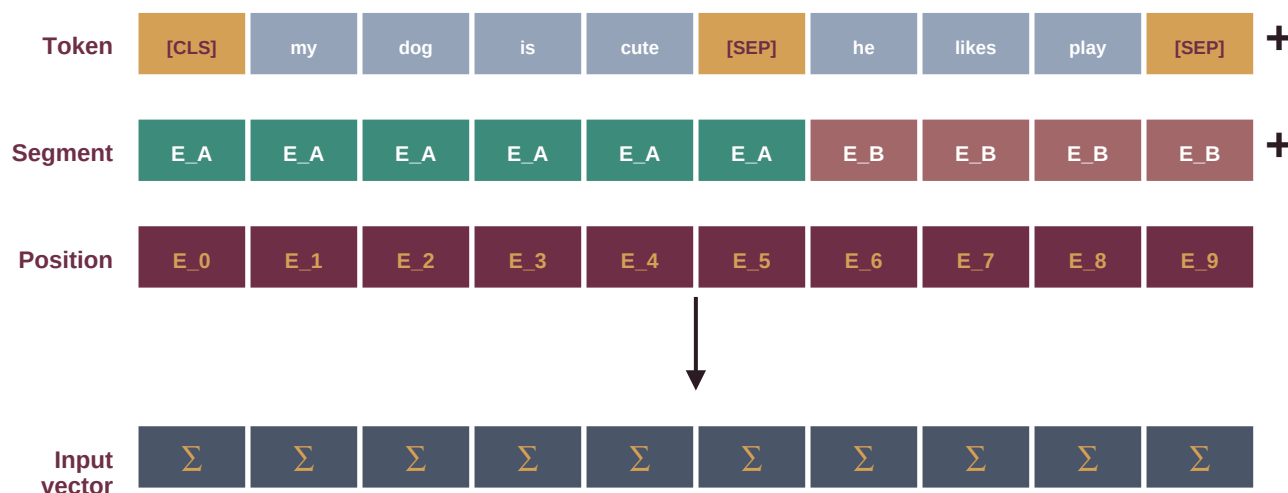
Used during MLM pretraining to mark positions whose original tokens should be predicted. At fine-tuning and inference time, [MASK] tokens generally don't appear — which is why the masking strategy (80/10/10) is designed to prepare the model for that.

[PAD] — the padding token

Used to make all sequences in a batch the same length. The attention mask is used to ensure these positions don't influence the model's computation. Mechanical detail, but important to know about.

4.4 The three embeddings

BERT input: three embeddings summed at every position



Two segments separated by [SEP]. Sentence A gets E_A; sentence B gets E_B.

Position embeddings are learned (not sinusoidal like the original transformer).

Figure 4.2 — BERT's input representation. Token, segment, and position embeddings are summed element-wise to produce the input vector at each position.

Token embeddings

Each token in BERT's vocabulary (about 30,000 entries for the English model) has a learned embedding vector. The vocabulary uses WordPiece tokenization, which splits rare words into subword pieces. For

example, "running" might become ["run", "###ning"]. This keeps the vocabulary manageable while still handling any word.

Segment embeddings

Only two values: E_A and E_B. Every token in sentence A gets E_A added to its embedding; every token in sentence B gets E_B. This tells the model which sentence each token belongs to. For single-sentence inputs, all tokens use E_A.

Position embeddings

Unlike the original Transformer's sinusoidal positional encoding, BERT uses **learned** position embeddings. Position 0 has its own vector, position 1 has another, and so on, up to position 511. These vectors are randomly initialized and learned during training.

This is the source of BERT's maximum sequence length of 512 tokens. Beyond position 511, no embedding exists. Modern variants (Longformer, BigBird, ModernBERT) extend this through various tricks.

5. Pretraining: How BERT Learned Its Knowledge

The pretraining setup mattered as much as the architecture. Devlin et al. trained BERT on a massive corpus for a long time, and that's where most of the model's capability came from.

5.1 The training corpus

BERT was trained on two text sources combined:

BooksCorpus — about 800 million words from 7,000+ self-published fiction books. Mostly long-form prose, useful for learning sentence relationships.

English Wikipedia — about 2.5 billion words. Article text only, with lists, tables, and headers stripped out.

Combined: about 3.3 billion words of clean, varied English text. This was a lot by 2018 standards. By modern standards, it's modest — GPT-3 was trained on roughly 100× more text.

5.2 The training process

BERT was trained for 1 million steps with batch size 256, which works out to about 40 epochs over the combined corpus. The training used 16 TPU chips for BERT-Base (about 4 days) and 64 TPU chips for BERT-Large (about 4 days).

The optimizer was Adam with a learning rate of 1e-4, linear warmup over the first 10,000 steps, then linear decay. Standard transformer training configuration.

5.3 What the model learned

By the end of pretraining, BERT had absorbed an enormous amount of linguistic and world knowledge implicitly:

Syntax. What words can follow which other words. What grammatical roles words play. How phrase structure works.

Semantics. What words mean in context. How meaning composes. How synonyms relate.

World knowledge. Common facts (Paris is in France, dogs bark, water is liquid). Common knowledge encoded in word co-occurrence patterns.

Pragmatics and coreference. Which pronoun refers to which noun. How discourse flows across sentences.

None of this was explicitly programmed. All of it emerged from the simple objective of "fill in the blanks" applied across billions of examples.

THE BIG IDEA, REPHRASED

Generic, task-agnostic pretraining can produce a model that captures most of what's hard about language understanding. Once you have such a model, specializing it for any specific task becomes much easier — you're not teaching it language anymore, you're just teaching it which patterns matter for your problem. This is the transfer-learning principle that NLP had been missing.

6. Fine-Tuning: The Paradigm BERT Defined

Pretraining gives you a model that understands language in general. Fine-tuning specializes it for a particular task. BERT's fine-tuning recipe became the template that every NLP project used from 2018 onward.

6.1 The basic recipe

Take a pretrained BERT. Add a small **task-specific head** — typically one or two linear layers on top of either the [CLS] output or the per-token outputs. Initialize the head randomly. Train the whole model (BERT + head) on the target task's labeled data, with a small learning rate, for a few epochs.

That's it. The same recipe works across an enormous range of tasks. Only the shape of the head and the loss function change.

One pretrained BERT, many task-specific fine-tuned heads

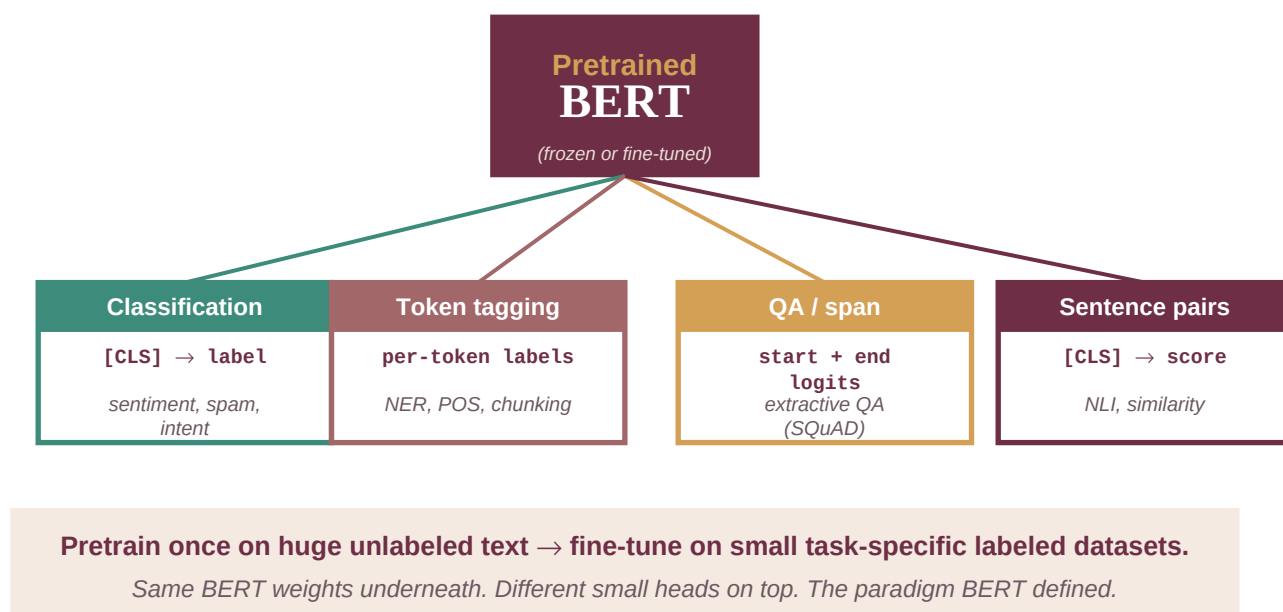


Figure 6.1 — The four canonical fine-tuning patterns from the BERT paper. Same underlying pretrained BERT in every case; different heads for different tasks.

6.2 Four standard patterns

Pattern 1 — Sentence classification

For tasks like sentiment analysis, spam detection, or intent classification: feed the sentence into BERT (preceded by [CLS]). Take the [CLS] output vector. Pass it through a single linear layer with one output per class. Softmax. Cross-entropy loss. Trains in minutes on a small dataset.

Pattern 2 — Token-level tagging

For tasks like named entity recognition or part-of-speech tagging: feed the sentence into BERT. Take every per-token output vector. Pass each through a small linear layer that predicts the tag for that token. Loss is summed across all positions. The bidirectional context is especially helpful here because deciding whether "Washington" is a person or a place depends on surrounding words.

Pattern 3 — Question answering / span prediction

For tasks like SQuAD: input is the question followed by the passage, separated by [SEP]. BERT predicts two probabilities per token: "is this where the answer starts?" and "is this where the answer ends?". At inference, pick the span with the highest combined probability. This pattern works because BERT can attend bidirectionally to relate the question to the passage at every position.

Pattern 4 — Sentence pairs

For tasks like natural language inference (does premise entail hypothesis?) or semantic similarity (how similar are these two sentences?): feed both sentences into BERT separated by [SEP], with segment embeddings distinguishing them. Use the [CLS] output for classification, just like single-sentence classification but with two segments.

6.3 The GLUE results

BERT was evaluated on GLUE — the General Language Understanding Evaluation benchmark — a collection of 9 diverse NLP tasks (sentiment, entailment, paraphrase detection, similarity, etc.). The results were a watershed moment:

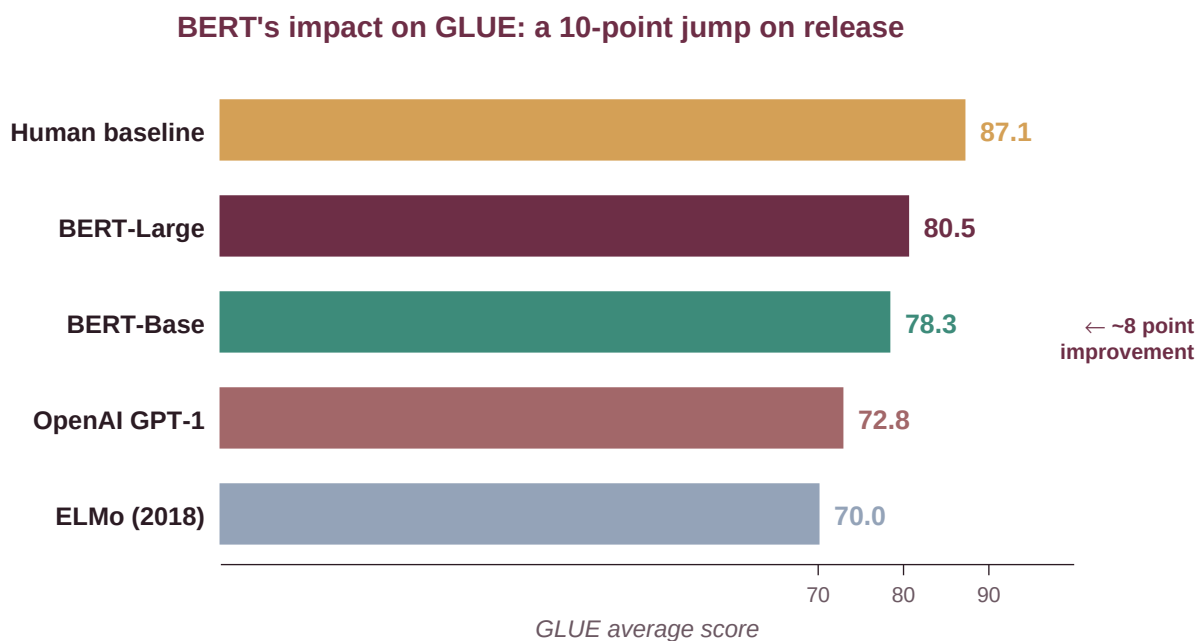


Figure 6.2 — BERT's GLUE results vs. prior state of the art. The 8-point improvement was an unprecedented jump on a mature benchmark, and it triggered the modern transfer-learning era in NLP.

Before BERT, GLUE scores had been creeping up by 1-2 points per year as people engineered better task-specific architectures. BERT-Large jumped the field by about 8 points overnight, with a generic model and almost no task-specific tricks. Within a year, BERT-style approaches dominated essentially every NLP

benchmark.

6.4 Why fine-tuning works so well

Three reasons:

- 1. The hard part is already done.** Pretraining taught the model language. Fine-tuning just teaches it to map language to a specific output. The task-specific signal can be small because most of the work is already done.
- 2. The model has the right inductive biases.** The transformer's attention mechanism turns out to be a remarkably general way to model language. The pretrained model is in a good region of parameter space, and fine-tuning makes small adjustments.
- 3. Small task datasets are enough.** A few thousand labeled examples is often plenty when starting from pretrained weights. Compared to the millions of examples that would be needed to train from scratch, this is a transformative reduction.

7. Code Walkthrough

Using BERT in practice is straightforward thanks to the HuggingFace transformers library. We'll walk through three things: loading BERT, computing embeddings, and fine-tuning for a classification task.

7.1 Setup

```
pip install transformers torch datasets
```

7.2 Loading BERT and computing embeddings

```
from transformers import AutoTokenizer, AutoModel
import torch

# Load pretrained BERT
tokenizer = AutoTokenizer.from_pretrained("bert-base-uncased")
model = AutoModel.from_pretrained("bert-base-uncased")
model.eval()

# Tokenize a sentence
text = "The cat sat on the mat."
inputs = tokenizer(text, return_tensors="pt")
# inputs is a dict with:
#   input_ids:      tensor of token IDs (incl. [CLS] at start, [SEP] at end)
#   attention_mask: 1 for real tokens, 0 for padding
#   token_type_ids: segment IDs (0 for single-sentence input)

# Run through BERT
with torch.no_grad():
    outputs = model(**inputs)

# Two main outputs:
last_hidden = outputs.last_hidden_state # (1, seq_len, 768) – per-token vectors
pooler_out  = outputs.pooler_output     # (1, 768) – [CLS] passed through a tanh layer

# Common pattern: use [CLS] vector as sentence embedding
cls_embedding = last_hidden[:, 0, :]    # (1, 768)
print(cls_embedding.shape)              # torch.Size([1, 768])
```

The `last_hidden_state` gives you one 768-dimensional vector per input token. The first one (index 0) corresponds to [CLS] and is the standard "sentence representation." The other positions give per-token contextual representations.

7.3 Fine-tuning for sentiment classification

```
from transformers import (
    AutoTokenizer, AutoModelForSequenceClassification,
    TrainingArguments, Trainer,
)
from datasets import load_dataset

# Load BERT with a randomly-initialized classification head
model = AutoModelForSequenceClassification.from_pretrained(
    "bert-base-uncased",
    num_labels=2,      # binary sentiment
```

```

)
tokenizer = AutoTokenizer.from_pretrained("bert-base-uncased")

# Load a labeled sentiment dataset (e.g., IMDb movie reviews)
dataset = load_dataset("imdb")

def tokenize(batch):
    return tokenizer(batch["text"], truncation=True, max_length=256, padding="max_length")

dataset = dataset.map(tokenize, batched=True)
dataset = dataset.rename_column("label", "labels")
dataset.set_format("torch", columns=["input_ids", "attention_mask", "labels"])

# Standard fine-tuning hyperparameters
args = TrainingArguments(
    output_dir="./bert-imdb",
    learning_rate=2e-5,          # small LR for fine-tuning
    per_device_train_batch_size=16,
    num_train_epochs=3,
    weight_decay=0.01,
    evaluation_strategy="epoch",
)

trainer = Trainer(
    model=model,
    args=args,
    train_dataset=dataset["train"],
    eval_dataset=dataset["test"],
)
trainer.train()

```

That's a complete sentiment classifier fine-tuning script in about 30 lines. The model usually reaches 92-94% accuracy on IMDb in a few hours on a single GPU. Notice the small learning rate (2e-5) — critical for fine-tuning, since you don't want to disturb the pretrained weights too aggressively.

7.4 Using BERT for embeddings (sentence-transformers)

For many production use cases (semantic search, clustering, RAG retrieval), you want a single embedding vector per sentence. Raw BERT's [CLS] vector works okay but isn't great. The **sentence-transformers** library provides BERT-based models fine-tuned specifically to produce high-quality sentence embeddings:

```

from sentence_transformers import SentenceTransformer
import numpy as np

# Load a BERT-based sentence embedding model
model = SentenceTransformer("sentence-transformers/all-MiniLM-L6-v2")

sentences = [
    "The cat sat on the mat.",
    "A feline was on the rug.",
    "The stock market rose 3% today.",
]

embeddings = model.encode(sentences)
# Each is a 384-dim vector

# Similarity between first two sentences (similar meaning) should be high
sim_12 = np.dot(embeddings[0], embeddings[1])
# Between 1 and 3 (unrelated) should be low

```

```
sim_13 = np.dot(embeddings[0], embeddings[2])

print(f"cat~feline: {sim_12:.3f}")    # ~0.65
print(f"cat~stocks: {sim_13:.3f}")    # ~0.05
```

Models like all-MiniLM-L6-v2, MPNet, and BGE are the workhorses of modern retrieval. They're BERT-style encoders specialized for producing good sentence embeddings via additional training on sentence-similarity tasks. If you've used vector search with OpenAI's text-embedding-3-small, Cohere embeddings, or Voyage embeddings, you've used something in this family.

8. Limitations & The BERT Family

BERT had real limitations — some recognized at the time, others discovered later. Each one spawned variants that addressed it.

8.1 Limitation: NSP isn't useful

RoBERTa (Liu et al., Meta, 2019) was the first major BERT critique. The authors took the original BERT recipe and methodically varied each hyperparameter, discovering that:

- NSP didn't help — removing it improved downstream performance.
- BERT was undertrained — more steps and more data helped substantially.
- Dynamic masking (different masks each epoch) helped slightly.
- Larger batches and longer sequences helped.

RoBERTa with these changes outperformed BERT-Large on essentially every benchmark. The lesson: BERT was a great paper but a slightly underbaked recipe.

8.2 Limitation: too expensive to deploy

BERT-Base has 110M parameters. Running it on every search query at Google scale is expensive. Several variants tackled this:

DistilBERT (Sanh et al., HuggingFace, 2019). Trained a smaller (66M param) model to mimic BERT's behavior via knowledge distillation — the smaller model learns to match BERT's outputs on the same inputs. Result: ~60% faster, ~40% smaller, ~97% of BERT's quality. Still widely used today.

ALBERT (Lan et al., Google, 2019). "A Lite BERT." Two key tricks: share weights across all transformer layers (one shared layer reused 12 times), and factorize the embedding matrix. Drastically reduces parameter count while keeping similar quality.

MobileBERT (Sun et al., 2020). Specifically designed for mobile deployment. Even smaller and faster than DistilBERT, designed to run on phones.

8.3 Limitation: MLM is inefficient

Each MLM example only generates training signal for 15% of tokens — the masked ones. The other 85% are wasted, computationally. ELECTRA (Clark et al., Google, 2020) replaced this with a more efficient alternative.

ELECTRA's idea: replaced token detection

Instead of masking tokens, ELECTRA uses a small "generator" model to replace some tokens with plausible-but-wrong alternatives. Then it trains a "discriminator" model (the actual model we'll fine-tune) to detect which tokens were replaced. Now every token contributes to the training signal, not just 15%. The result is a more sample-efficient pretraining objective.

8.4 Limitation: attention scales $O(N^2)$

BERT's max sequence length is 512 tokens because attention is $O(N^2)$. Longer sequences become prohibitively expensive.

Longformer (Beltagy et al., 2020) and **BigBird** (Zaheer et al., 2020) use sparse attention patterns (local windows + a few global tokens) to handle much longer sequences, up to 4096+ tokens, with $O(N)$ complexity.

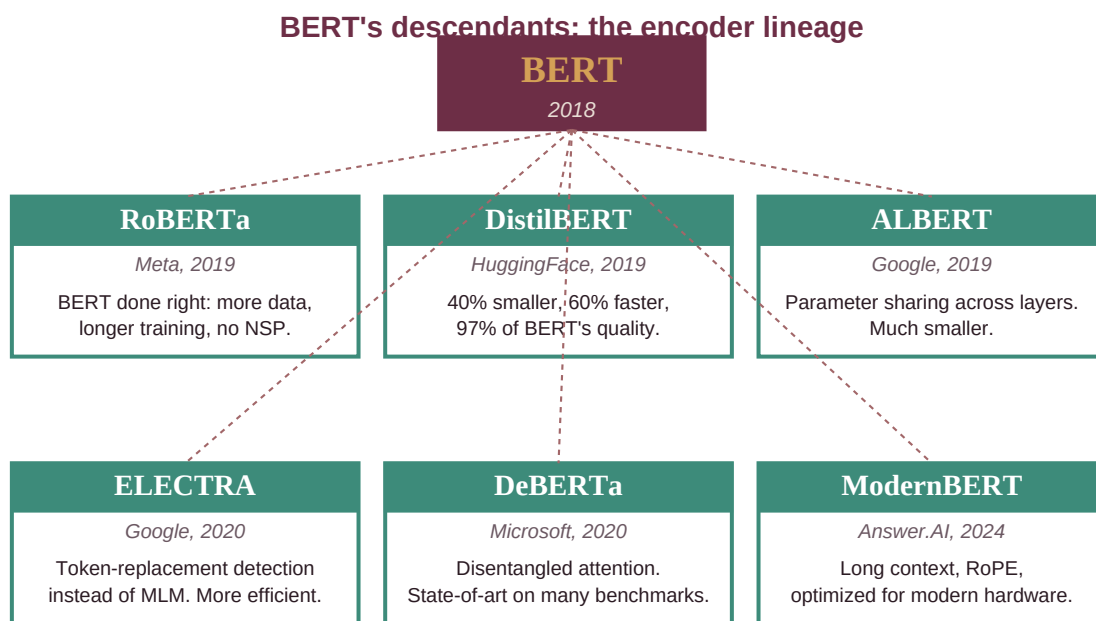
ModernBERT (Answer.AI + LightOn, late 2024) is the most recent major update. Brings transformer best practices from the LLM era to BERT: rotary positional embeddings, FlashAttention, longer context (8192 tokens), better data. Drop-in replacement for BERT with substantially better quality and longer context. Worth knowing about for any production BERT use in 2026.

8.5 Limitation: bidirectionality precludes generation

BERT cannot generate text. The bidirectional attention means it doesn't have a natural way to do autoregressive prediction. For tasks like translation, summarization, or open-ended generation, you need decoder or encoder-decoder models. This is fine for what BERT is for — understanding tasks — but it's a fundamental architectural choice.

Encoder-decoder hybrids like **T5** (Raffel et al., 2020) and **BART** (Lewis et al., 2020) combine BERT-like encoding with GPT-like generation, useful for tasks needing both understanding and generation.

8.6 The family tree



Every modern embedding model and search re-ranker is in this family.

BERT-style encoders quietly power half of production NLP today.

Figure 8.1 — BERT's major descendants. Each addresses a specific limitation of the original. The lineage is still actively evolving — ModernBERT in 2024 brought modern transformer optimizations to encoder models.

8.7 The DeBERTa improvements

DeBERTa (He et al., Microsoft, 2020) deserves its own callout. It introduced two innovations that meaningfully improved encoder model quality:

Disentangled attention. The original transformer encodes position by adding position embeddings to token embeddings. DeBERTa instead represents position separately and computes attention as a sum of content-to-content, content-to-position, and position-to-content terms. More expressive, modestly better quality.

Enhanced mask decoder. A small architectural change to the MLM head that gives access to absolute position information.

DeBERTa-v3 (with ELECTRA-style training) became the state of the art on GLUE and is still competitive in 2026 for many encoder use cases.

9. Adoption: Where BERT Lives Today

BERT-style encoders are everywhere in production NLP — just usually invisible. Here's a tour.

9.1 Search ranking

Google announced in 2019 that BERT was being used to improve search results, particularly for understanding the intent of natural-language queries. Microsoft Bing made similar announcements. Both companies have continued evolving these deployments with newer encoder models.

Typical deployment: a smaller, distilled BERT-style model serves as a **re-ranker**. The search system first uses traditional methods (BM25, fast embedding retrieval) to find ~1000 candidate documents, then the BERT re-ranker scores each candidate against the query to produce the final ranking. The BERT-style model adds expensive but accurate semantic understanding on top of fast but shallow keyword matching.

9.2 Sentence embeddings for retrieval

Every Retrieval-Augmented Generation (RAG) system needs an embedding model to encode documents and queries into vectors that can be searched. The dominant models in this space are BERT descendants:

- **sentence-transformers** family (MiniLM, MPNet, etc.) — the open-source workhorse for embedding-based retrieval.
- **BGE (BAAI General Embedding)** models — BERT-style encoders specifically tuned for retrieval. Top of the MTEB leaderboard for several stretches in 2023-2024.
- **E5, GTE, Jina embeddings** — various BERT-descended embedding models with different speed/quality tradeoffs.
- **Cohere, Voyage, OpenAI embeddings** — commercial embedding APIs. Architectures aren't public but are widely understood to be BERT-style encoders.

9.3 Named entity recognition and information extraction

Most production NER systems use BERT-style encoders with a token-classification head. spaCy, Hugging Face Spaces models, Azure Cognitive Services, Amazon Comprehend — all use BERT-family models for entity recognition under the hood. The bidirectional context is especially valuable here because entity boundaries and types often depend on words after the entity.

9.4 Content moderation and safety

Detecting toxic content, hate speech, spam, and policy violations at scale requires fast, accurate classifiers. Most major platforms use BERT-style encoders for this. The combination of strong accuracy and reasonable inference cost is ideal for production moderation systems running on every user post.

9.5 Customer support and intent classification

When you submit a customer support ticket or call a phone tree, the system almost certainly uses a BERT-style classifier to route your request. Intent classification, sentiment routing, urgency detection, language detection — all standard BERT fine-tuning use cases. Tools like Rasa, Dialogflow, and many enterprise NLU platforms have BERT models behind their APIs.

9.6 Biomedical and specialized domains

Domain-specific BERT variants are common in research and specialized industry:

BioBERT, ClinicalBERT, PubMedBERT — trained on biomedical text. Used for clinical NLP, literature mining, drug-protein interaction extraction.

LegalBERT — trained on legal text. Used for contract analysis, case law retrieval, regulatory compliance.

FinBERT — trained on financial text. Used for sentiment in earnings calls, news event extraction, risk classification.

SciBERT — trained on scientific papers. Used for citation prediction, paper similarity, scientific entity recognition.

The pattern: take BERT (or a descendant like ModernBERT), continue pretraining on your domain's text, then fine-tune for specific tasks. The domain pretraining step substantially improves quality for specialized vocabulary and concepts.

9.7 Why not just use an LLM?

Modern LLMs (GPT-4, Claude, Gemini) can handle many tasks BERT was historically used for — classification, NER, embeddings (via LLM-based embedding models). Why are encoder models still everywhere?

Cost. A BERT-base classifier costs ~\$0.0001 per inference. Running the same task through GPT-4 costs ~\$0.01-0.10 per call — 100-1000× more.

Latency. BERT inference is 10-100ms. LLM API calls are 500-5000ms. For anything user-facing or in a tight loop, the difference matters.

Determinism. A fine-tuned BERT model produces the same output every time. LLMs sample stochastically; getting them to behave deterministically is tricky.

Privacy. Self-hosted BERT models keep data inside your infrastructure. Sending everything to OpenAI or Anthropic isn't always acceptable.

Result: most production NLP pipelines use a hybrid — BERT-style encoders for cheap, fast, high-volume tasks (classification, retrieval, NER, filtering), with LLMs reserved for the parts that actually need them (open-ended reasoning, generation, complex synthesis).

10. Worked Examples

Three concrete production scenarios where BERT-style models are the right answer.

10.1 Example: Building a RAG system

Scenario

You're building a customer support assistant that answers questions from your company's documentation. The docs total ~50,000 pages. When a user asks a question, you need to find the most relevant pages quickly and pass them to an LLM for the final answer.

BERT's role

Use a sentence-transformers BERT model to encode every paragraph in the docs into a 384-dim vector. Store these in a vector database (Pinecone, Weaviate, Qdrant, etc.). At query time:

1. Encode the user's question into a vector with the same model.
2. Vector-search to find the top-20 most similar paragraphs.
3. Optionally use a BERT-style cross-encoder (like ms-marco-MiniLM-L-6-v2) to re-rank these 20 candidates more accurately.
4. Pass the top 3-5 paragraphs as context to Claude or GPT-4 to generate the final answer.

Result

All embedding work runs on a small GPU or even CPU. Vector search returns in ~50ms. Re-ranking adds ~100ms. The LLM call takes 1-3 seconds. Total user-facing latency: under 4 seconds for an accurate, source-grounded answer. Cost: a few cents per query, mostly from the LLM. Without BERT-based retrieval, you'd either send all 50,000 pages to the LLM (impossible) or use keyword search (much lower quality).

10.2 Example: High-volume spam classification

Scenario

You run a community platform with 10 million user posts per day. You need to filter spam in real time, before it reaches other users. Budget: under \$500/day for moderation infrastructure.

BERT's role

Fine-tune DistilBERT (smaller, faster than full BERT) on 50,000 labeled examples of spam vs. legitimate posts. Deploy the fine-tuned model on a single GPU with batching. Run every post through it.

Numbers

DistilBERT inference: ~5ms per post with batch size 32. A single A10 GPU handles ~100 posts/second sustained — 8.6M/day capacity per GPU. Two GPUs comfortably handle the 10M/day target with headroom. GPU cost: ~\$1.50/hour each × 24 hours × 2 = ~\$72/day. Easily within budget.

Compare to using GPT-4 for the same task: ~\$0.001 per call × 10M = \$10,000 per day. 140× more expensive. The BERT solution is the obvious choice for high-volume, well-defined classification.

10.3 Example: Medical entity extraction

Scenario

You work at a health-tech company. You have 500,000 clinical notes from providers, written in messy clinical English. You need to extract all mentions of medications, dosages, conditions, and procedures, in a structured format that can be queried.

BERT's role

Take BioBERT (BERT pretrained on biomedical literature) as the starting point. Fine-tune on a few thousand expert-labeled clinical notes for token-level entity classification with tags like MEDICATION, DOSAGE, CONDITION, PROCEDURE. Use BIO tagging (Begin/Inside/Outside) to handle multi-word entities.

Why BERT here and not an LLM?

Privacy: PHI (protected health information) can't be sent to external APIs without significant compliance work. Self-hosted BERT keeps data inside the secure environment.

Structured output: Token-tagging produces clean structured data — exact character spans, exact entity types. LLMs are flexible but less consistent at this kind of structured extraction.

Domain knowledge: BioBERT's pretraining on millions of biomedical papers gives it strong priors for clinical vocabulary. A generic LLM would need careful prompting to match this.

Volume: 500K documents to process. Batch-processing through BERT is much cheaper than running them all through an LLM.

11. Likely Interview Questions

If you understand the previous chapters, you can answer all of these.

Q: What's the key idea behind BERT?

A: Bidirectional pretraining for language understanding. Instead of predicting the next word (which forces left-to-right context), BERT randomly masks 15% of input tokens and trains the model to predict them from both left and right context. This lets the model build representations that use the full sentence rather than just what came before. Combined with the Transformer encoder architecture and pretraining on large unlabeled corpora, this produces a model that can be fine-tuned on small labeled datasets to achieve strong results across many NLP tasks.

Q: Why can't BERT generate text like GPT can?

A: BERT uses bidirectional self-attention — every position can attend to every other position. There's no causal mask preventing attention to future positions. This is great for understanding (where you want full context) but incompatible with autoregressive generation, which requires that when predicting token t , the model only sees tokens $1..t-1$. For generation, you need a decoder-style model with causal masking, like GPT, or an encoder-decoder model like T5.

Q: What does the [CLS] token do?

A: [CLS] is a special token prepended to every BERT input. Its output vector after passing through the model is used as a single "summary" representation of the entire input sequence. For classification tasks, you put a small linear head on top of the [CLS] vector. For sentence-pair tasks (like entailment or similarity), [CLS] sees both sentences and represents their relationship. The model learns to make [CLS]'s output useful as a global summary during pretraining (via the NSP task) and fine-tuning.

Q: How does BERT's masking strategy work exactly?

A: BERT picks 15% of input tokens uniformly at random. Of those, 80% are replaced with [MASK], 10% are replaced with a random other token, and 10% are left unchanged. In all three cases, the model is asked to predict the original token at that position. The random-replacement and unchanged cases prevent the model from only learning to handle [MASK] tokens. Since [MASK] doesn't appear at fine-tuning or inference time, the model needs to maintain good representations for every token, not just masked ones.

Q: Is Next Sentence Prediction actually useful?

A: Mostly no. The original BERT paper introduced NSP as a second pretraining objective to help with sentence-pair understanding tasks. Later analysis (notably RoBERTa) showed that NSP doesn't actually help and may slightly hurt. The signal is too easy to learn — the model just looks at topic similarity between the sentences, which doesn't teach meaningful sentence-level reasoning. Most modern BERT-family models (RoBERTa, DeBERTa, ELECTRA, ModernBERT) drop NSP entirely. But the original BERT paper used it, so it's part of the canonical description.

Q: How does BERT handle word order if it uses bidirectional attention?

A: Through position embeddings. BERT adds a learned position-embedding vector to each token's embedding before any attention happens. Position 0 has its own vector, position 1 has another, and so on, up to 511 (BERT's max sequence length). This is different from the original Transformer's fixed sinusoidal encoding — BERT learns the position vectors during pretraining. Without position embeddings, the bidirectional self-attention would be permutation-invariant and unable to distinguish word order.

Q: What are the differences between BERT-Base and BERT-Large?

A: Both use the same architecture; they differ in scale. BERT-Base has 12 transformer layers, 768 hidden dimensions, 12 attention heads, totaling 110M parameters. BERT-Large has 24 layers, 1024 hidden dim, 16 heads, totaling 340M parameters. BERT-Large performs better on most benchmarks but costs ~3× more memory and ~3× more compute. For most production use cases, BERT-Base or DistilBERT (66M params) is plenty.

Q: How would you use BERT for sentence similarity?

A: There are two main approaches. First, the 'cross-encoder' approach: feed both sentences into BERT as a pair (separated by [SEP]), and use the [CLS] output to predict a similarity score. Very accurate but expensive — you have to run BERT once per pair. Second, the 'bi-encoder' approach used by sentence-transformers: encode each sentence independently into a fixed embedding vector, then compute cosine similarity between the vectors. Much faster (you can pre-compute embeddings once and search them), slightly less accurate. Production retrieval systems often use bi-encoders for first-stage retrieval and cross-encoders for re-ranking.

Q: What is WordPiece tokenization?

A: A subword tokenization scheme used by BERT. The vocabulary contains about 30,000 entries: common words and word pieces. Rare or unseen words are split into known pieces. For example, "running" might tokenize to ["run", "##ning"] where "##" marks a continuation. This handles out-of-vocabulary words gracefully (unlike word-level tokenizers) while keeping the vocabulary manageable (unlike character-level tokenizers). Modern variants include BPE (Byte-Pair Encoding) used by GPT and SentencePiece used by T5 and Llama. The principles are similar.

Q: When would you choose BERT over an LLM in 2026?

A: Four scenarios. First, high-volume classification or extraction tasks where BERT's 10-100× lower inference cost matters. Second, latency-sensitive applications where 10-50ms response times beat 500-3000ms LLM calls. Third, embedding-based retrieval for RAG — BERT-style models are the standard. Fourth, privacy-sensitive deployments where you need to self-host. The rule of thumb: if the task is well-defined, narrow, and high-volume, use BERT. If it's open-ended, complex, or requires generation, use an LLM. Production systems typically combine both — BERT for the many small fast decisions, LLMs for the few that actually need reasoning.

Appendix A. Key Equations & Terms

Equations

Input representation

$$\text{input}_i = \text{token_emb}_i + \text{segment_emb}_i + \text{position_emb}_i$$

Three embeddings summed at every position to produce the model's input.

MLM loss

$$L_{\text{MLM}} = -\sum_{(i \in \text{masked})} \log P(\text{token}_i \mid \text{context})$$

Cross-entropy over the masked positions only.

NSP loss

$$L_{\text{NSP}} = -\log P(\text{IsNext} \mid [\text{CLS}])$$

Binary classification on the [CLS] output. (Largely abandoned post-RoBERTa.)

Combined pretraining objective

$$L_{\text{total}} = L_{\text{MLM}} + L_{\text{NSP}}$$

Both losses added; both signals shape the same set of weights.

Bidirectional self-attention

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}(\mathbf{Q} \cdot \mathbf{K}^T / \sqrt{d_k}) \cdot \mathbf{V}$$

Same as the original Transformer encoder — no causal mask.

Classification fine-tuning

$$P(y) = \text{softmax}(\mathbf{W}_{\text{cls}} \cdot [\text{CLS}]_{\text{output}} + \mathbf{b})$$

A linear head on top of the [CLS] vector. Cross-entropy against the task labels.

Terms

MLM (Masked Language Modeling)

Pretraining objective: randomly mask 15% of input tokens and train the model to predict them from surrounding context. The bidirectional alternative to next-token prediction.

NSP (Next Sentence Prediction)

BERT's secondary pretraining objective: given two sentences, predict whether B actually followed A in the source corpus. Later shown to be of limited value; most modern variants skip it.

[CLS]

Special classification token prepended to every BERT input. Its output vector is used as a single sentence representation for classification heads.

[SEP]	Special separator token. Marks sentence boundaries in pair inputs and indicates the end of the sequence.
[MASK]	Special token used during MLM pretraining to indicate the position the model should predict. Not present at fine-tuning or inference time.
Segment embedding	An additional embedding added to each token that indicates which sentence (A or B) it belongs to. Helps the model handle sentence-pair inputs.
Position embedding (BERT-style)	A learned embedding for each position 0-511. Replaces the original Transformer's sinusoidal positional encoding.
WordPiece tokenization	Subword tokenization scheme used by BERT. About 30K vocabulary entries; rare words are split into multiple pieces marked with ##.
Encoder-only model	A transformer model that uses only the encoder side of the original architecture. BERT, RoBERTa, DeBERTa, etc. Optimized for understanding rather than generation.
Bi-encoder vs cross-encoder	Two architectures for sentence-pair tasks. Bi-encoder: each sentence independently encoded to a vector; similarity computed between vectors. Fast, slightly less accurate. Cross-encoder: both sentences fed in together; joint representation produced. Slow, more accurate. Production systems often use bi-encoders for retrieval and cross-encoders for re-ranking.
DistilBERT	A smaller, faster BERT created via knowledge distillation — a compressed model trained to mimic the larger one. 40% smaller, 60% faster, 97% of BERT's quality.
ModernBERT	The 2024 modernization of BERT: rotary position embeddings, FlashAttention, 8192-token context, better data. Drop-in replacement with substantially better quality for modern production use.

Further reading

Original paper: Devlin et al., "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding" (2018). arXiv:1810.04805.

RoBERTa: Liu et al., "RoBERTa: A Robustly Optimized BERT Pretraining Approach" (2019). arXiv:1907.11692. The most important follow-up; corrects several BERT recipe issues.

DistilBERT: Sanh et al., "DistilBERT, a distilled version of BERT" (2019). arXiv:1910.01108. The compressed BERT widely used in production.

ELECTRA: Clark et al., "ELECTRA: Pre-training Text Encoders as Discriminators Rather Than Generators" (2020). arXiv:2003.10555.

DeBERTa: He et al., "DeBERTa: Decoding-enhanced BERT with Disentangled Attention" (2020). arXiv:2006.03654.

ModernBERT: Warner et al., "Smarter, Better, Faster, Longer: A Modern Bidirectional Encoder" (2024). The state-of-the-art encoder model for production use in 2026.

Sentence-Transformers: Reimers & Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks" (2019). arXiv:1908.10084. The basis for nearly all production sentence-embedding work.

Companion guides: see the Attention Is All You Need guide for the underlying Transformer architecture, the LoRA guide for parameter-efficient fine-tuning (heavily applied to BERT models), and the LLM Evolution companion for where BERT fits in the broader history.